

Example: Disadvantages of Multiple Services Sharing Databases

1. Overview

When multiple services directly share the same database, problems appear not only technically - but also organizationally.

This becomes painful as teams grow.

2. Example Scenario: E-commerce Platform

Suppose we have:

Services:

- Orders Service
- Metrics Service

Orders Service Handles:

- Create orders
- Update orders
- Cancel orders
- Refunds

Metrics Service Handles:

Business analytics such as:

- Orders in last 7 days
- Revenue today
- Refund ratio
- Daily GMV

3. Problem 1: No Single Source of Truth

Different teams compute metrics differently.

Example:

“Total Orders in Last 7 Days”

Questions:

- Include cancelled orders?
- Include refunded orders?
- Use UTC or local timezone?
- Include current day?

Result:

- Conflicting dashboards
- Wrong decisions
- Endless meetings

4. Better Pattern

Create dedicated:

None

Metrics Service

This service owns metric definitions.

5. How Metrics Service Should Work

Instead of querying Orders DB directly:

None

Metrics API Request



Metrics Service



Orders Service API / Event Stream



Compute Metric



Return Result

6. Shared DB Problem: Schema Coupling

Suppose Orders team wants optimization.

Existing Table

SQL

Orders

Huge table with 5 years of data.

Slow queries.

Orders Team Proposal

Split into:

SQL

Orders_Current (Last 1 year)

Orders_Archive (Older data)

Benefits:

- Faster recent queries
- Smaller hot table
- Better indexing
- Cheaper storage

7. But Metrics Team Uses Same DB

Now Metrics team must rewrite all queries:

SQL

```
SELECT ... FROM Orders_Current
```

```
UNION ALL
```

```
SELECT ... FROM Orders_Archive
```

Result:

- Migration delays
- Team friction
- Dependency bottlenecks

8. Problem 3: Technology Lock-In

Orders service wants:

Cassandra

Because:

- High write throughput
- Horizontal scaling

Metrics service wants:

SQL

Because:

- Aggregations
- Joins
- Simpler analytics

Shared DB makes this impossible.

9. Core Principle: Service Owns Its Data

Each service should own storage.

None

Orders Service → Orders DB

Metrics Service → Metrics DB

10. Better Integration Patterns

Option A: APIs

None

Metrics Service → Orders Service API

Good for:

- Real-time small queries

Option B: Events (Best for Scale)

None

Orders Created

Orders Cancelled

Orders Refunded



Kafka



Metrics Service

Metrics keeps own store.

Option C: ETL / Warehouse

None

Orders DB → ETL → Data Warehouse

Good for reporting.

11. Interview Gold Statement

Say:

“Services should not share databases. They should share contracts—APIs or events.”

That is a very strong industry statement.

12. Organizational Scalability

Shared DB hurts:

- Independent releases
- Team autonomy
- Ownership clarity
- Velocity

13. Technical Scalability

Shared DB hurts:

- Performance tuning
- DB engine choices
- Partitioning strategies
- Access control

14. Security Problems

If many services use same DB:

- Broad credentials
- Excess permissions
- Hard auditing

15. Common Exception Cases

Sometimes temporary shared DB acceptable:

- Startup MVP
- Legacy monolith migration
- Read-only reporting replica

But long-term avoid it.

16. Interview Example Answer

If asked:

“Can Metrics service query Orders DB directly?”

Say:

Initially maybe through read replica for speed, but long-term I'd move to event-driven ingestion so Metrics owns its data model and avoids coupling.

Excellent answer.

17. Final Summary

- Shared DB creates coupling
- Schema changes become slow

- Teams block each other
- Different workloads need different DBs
- Prefer APIs, events, ETL
- Service should own its data

18. Memorable Line

Shared database = shared pain.